

Digital Signal Processing in VLSI Design

Homework 5 Fast Fourier Transform

Name : TODO
Department : Electrical Engineering
Student ID : B11107027
Instructor : TODO

May 2026

Contents

1	Introduction	2
2	MATLAB Floating-Point SDF FFT Verification (Step 1–2)	2
2.1	SDF FFT Output in Bit-Reversed Order	2
2.2	Bit-Reversal Reordering Check	3
3	Fixed-Point Word-Length Selection (Step 3)	4
3.1	SQNR Definition and Test Inputs	4
3.2	Stage-by-Stage Fractional Word-Length Sweep	4
4	SDF FFT Architecture and Control Signals (Step 4–5)	8
4.1	Processing Element Control	8
4.2	Complex Multiplier Control	8
5	Twiddle ROM32 Design (Step 6)	9
6	Bit-Reversal Ping-Pong Buffer Verification (Step 7)	10
7	32-Point FFT Behavioral Simulation (Step 8)	10
8	96-Sample Streaming Verification (Step 9)	12
9	Synthesis Result without Internal Pipeline (Step 10)	15
10	Pipelined FFT and Post-Synthesis Simulation (Step 11)	15
11	Conclusion	16

1 Introduction

This homework implements a 32-point radix-2 decimation-in-frequency (DIF) single-path delay feedback (SDF) FFT processor. The work flow starts from a MATLAB floating-point model, then builds a fixed-point model for SQNR-based word-length selection, and finally verifies the corresponding Verilog RTL by behavioral simulation.

The provided input file is `FFTInput32.mat`. The MATLAB variable used in this report is `FFTIn32`, which contains one 32-point complex time-domain input vector. The floating-point SDF output is generated in bit-reversed order, then reordered into normal frequency order for comparison with MATLAB `fft()`.

2 MATLAB Floating-Point SDF FFT Verification (Step 1–2)

2.1 SDF FFT Output in Bit-Reversed Order

The implemented MATLAB model follows the radix-2 DIF FFT flow graph. Therefore, after five stages, the output sequence is decimated in frequency and appears in bit-reversed order. Table 1 lists the frequency-domain index associated with each SDF output sample. The real and imaginary values are obtained from the floating-point SDF FFT model.

Table 1: Step 1: SDF FFT outputs in bit-reversed order.

SDF output order	Frequency index	SDF output
0	0	$-1 - j$
1	16	$-1 + j$
2	8	$1 - j$
3	24	$1 - j$
4	4	$-1 + j$
5	20	$1 - j$
6	12	$1 + j$
7	28	$1 - j$
8	2	$-1 - j$
9	18	$1 - j$
10	10	$-1 + j$
11	26	$-1 - j$
12	6	$1 - j$
13	22	$-1 + j$
14	14	$-1 + j$
15	30	$1 + j$
16	1	$-1 - j$
17	17	$-1 + j$
18	9	$1 + j$
19	25	$-1 + j$
20	5	$1 + j$
21	21	$1 - j$
22	13	$1 + j$

SDF output order	Frequency index	SDF output
23	29	$-1 + j$
24	3	$1 - j$
25	19	$1 - j$
26	11	$-1 + j$
27	27	$1 - j$
28	7	$1 + j$
29	23	$-1 - j$
30	15	$1 - j$
31	31	$1 + j$

2.2 Bit-Reversal Reordering Check

The bit-reversal module maps the SDF output position to the normal frequency index by reversing the 5-bit binary representation. After reordering, the result is compared against MATLAB `fft(FFTIn32)`. Figure 1 shows the real and imaginary parts of the reordered FFT output, and Figure 2 shows the absolute error.

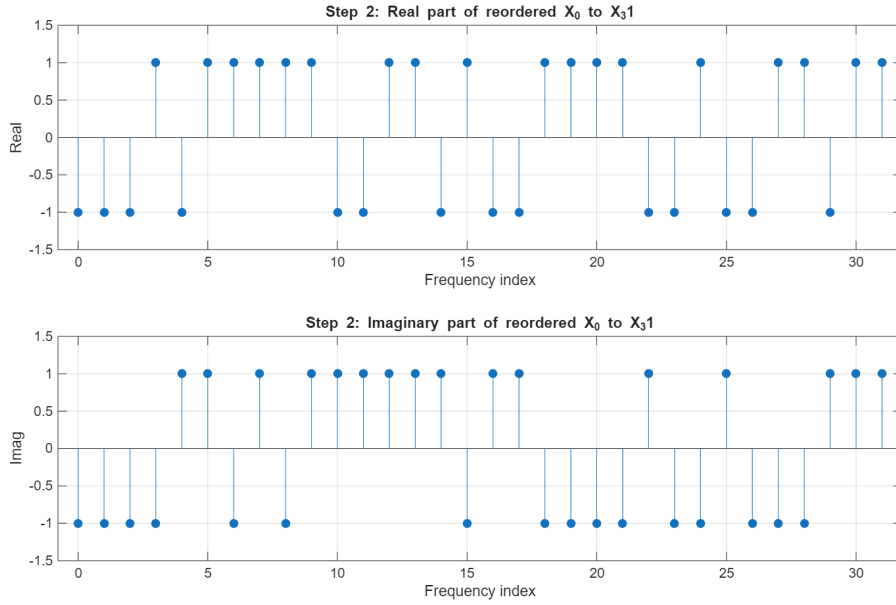


Figure 1: Step 2: Real and imaginary parts of reordered X_0 to X_{31} .

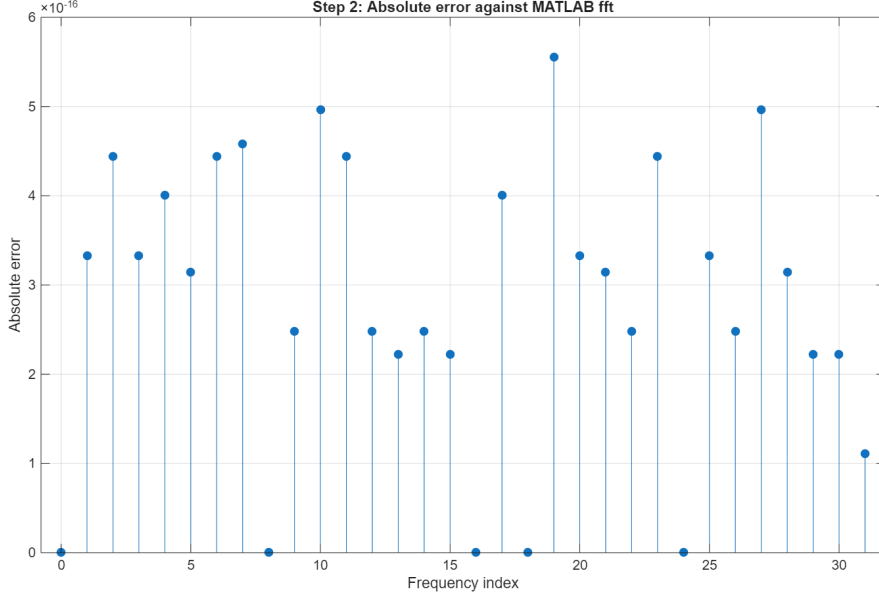


Figure 2: Step 2: Absolute error between reordered SDF FFT and MATLAB `fft()`.

The maximum floating-point error is

$$\max_i |X_i^{\text{SDF}} - X_i^{\text{MATLAB}}| = 5.551 \times 10^{-16},$$

which is much smaller than the required 10^{-10} .

3 Fixed-Point Word-Length Selection (Step 3)

3.1 SQNR Definition and Test Inputs

The SQNR requirement of the FFT processor is 35 dB. The SQNR is computed as

$$SQNR = 10 \log_{10} \left(\frac{E\{|X_i|^2\}}{E\{|\hat{X}_i - X_i|^2\}} \right),$$

where X_i is the reference floating-point FFT output and $\hat{X}_i$ is the quantized FFT output.

For the 96-sample test stream, three 32-point frequency-domain symbols are generated randomly from

$$\{1 + j, 1 - j, -1 + j, -1 - j\}.$$

Each symbol is transformed to the time domain by MATLAB `ifft()`, then fed into the fixed-point SDF FFT model.

3.2 Stage-by-Stage Fractional Word-Length Sweep

The fractional word-length is determined sequentially from Stage 1 to Stage 5. When selecting the word-length of a stage, all previously selected stages are fixed, and the current stage is swept from 9 to 18 fractional bits. The selected fixed-point truncation policy is truncation toward zero.

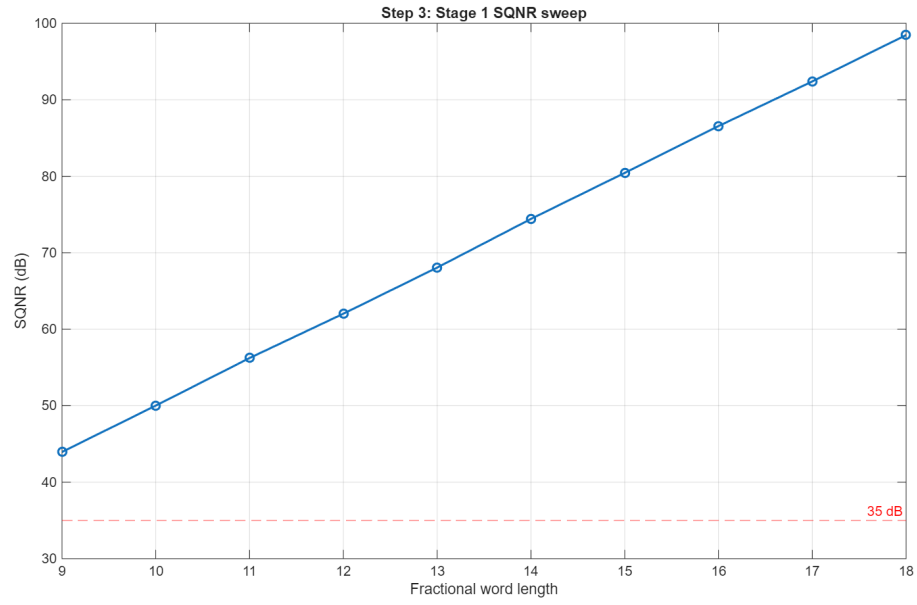


Figure 3: Step 3: SQNR vs. fractional word-length for Stage 1.

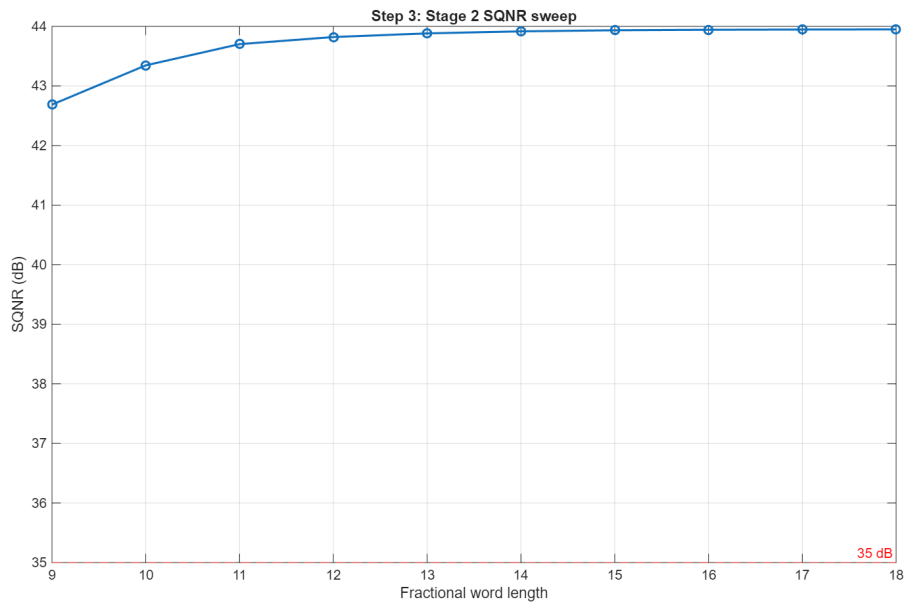


Figure 4: Step 3: SQNR vs. fractional word-length for Stage 2.

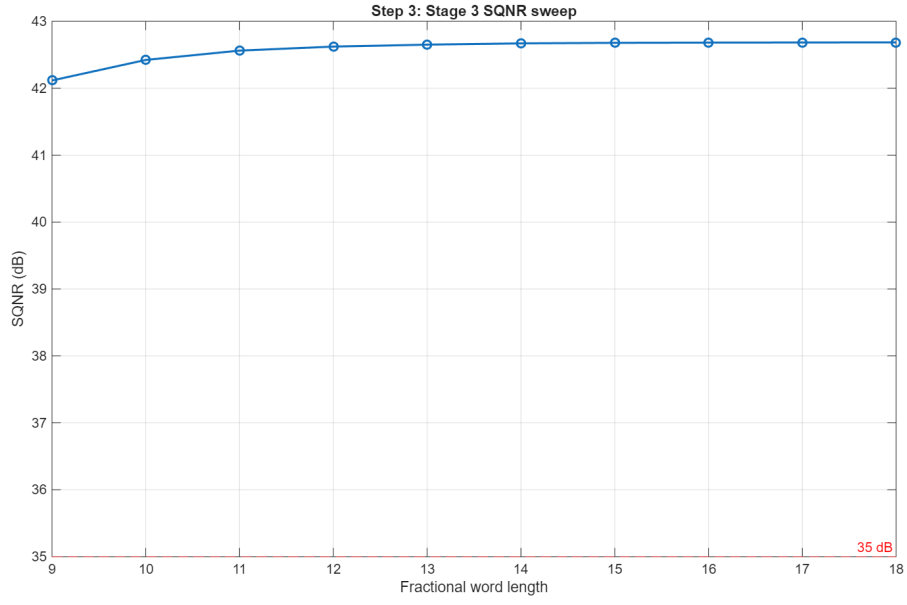


Figure 5: Step 3: SQNR vs. fractional word-length for Stage 3.

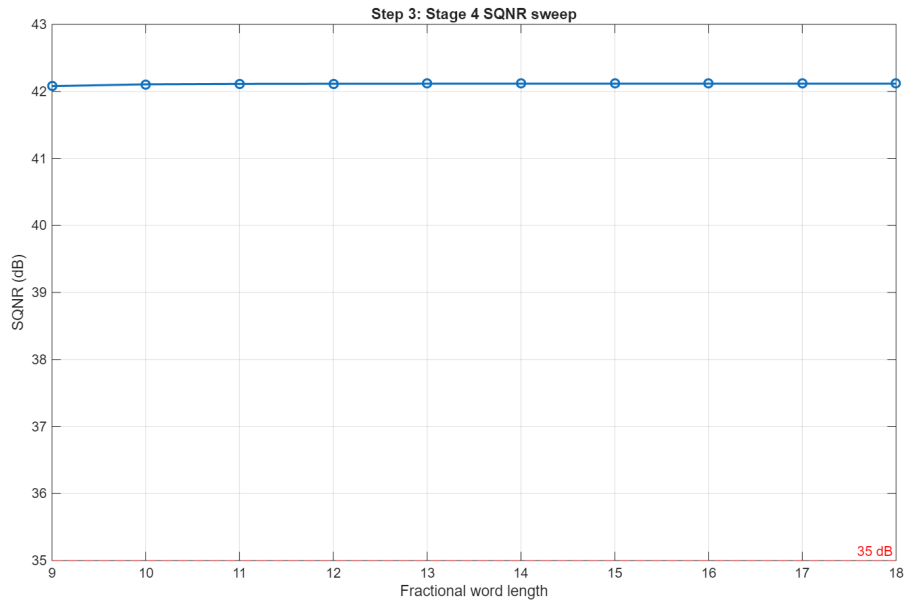


Figure 6: Step 3: SQNR vs. fractional word-length for Stage 4.

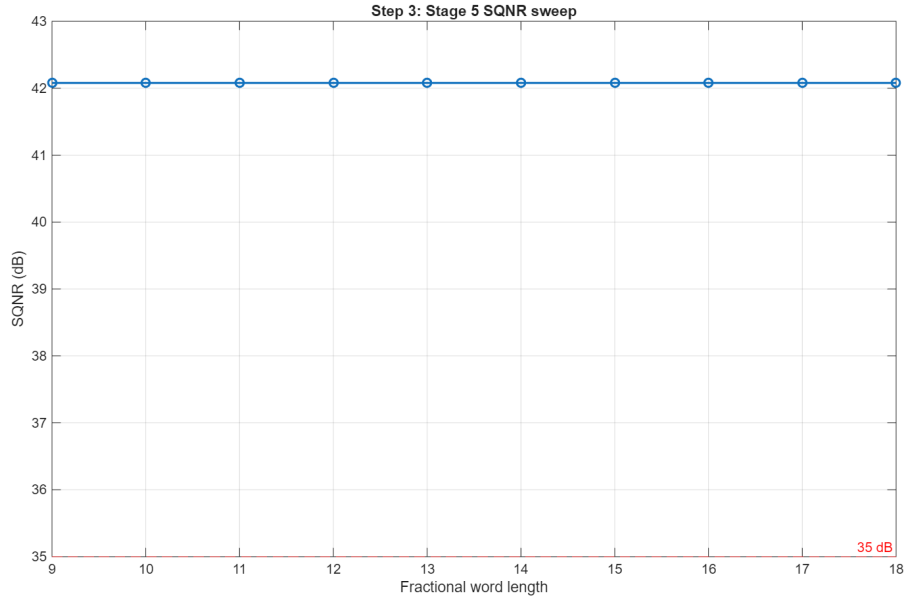


Figure 7: Step 3: SQNR vs. fractional word-length for Stage 5.

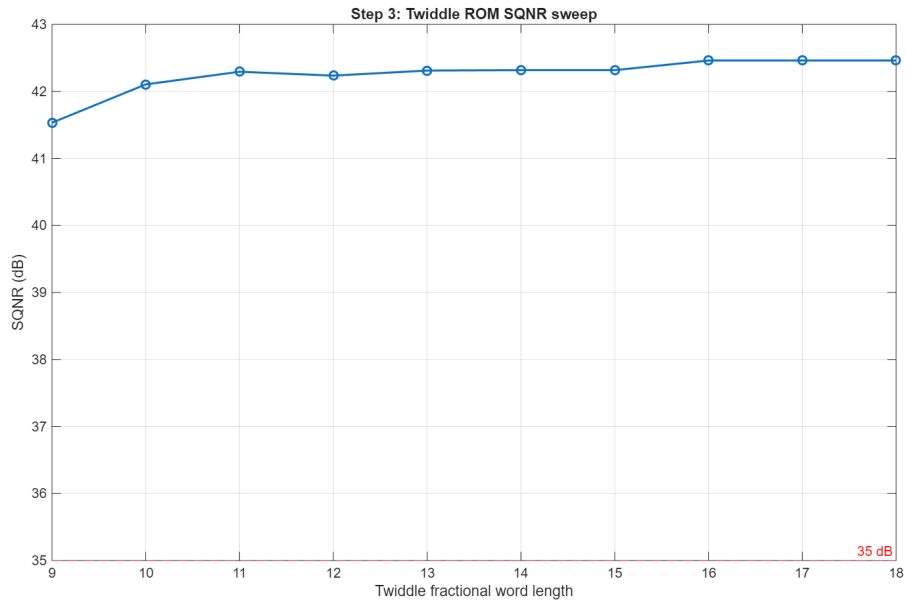


Figure 8: Step 3: SQNR vs. fractional word-length for the twiddle factors.

The final selected parameters are summarized in Table 2. All selected settings satisfy the 35 dB SQNR requirement.

Table 2: Selected fixed-point parameters.

Parameter	Value
Stage fractional bits $(w_{f,1}, \dots, w_{f,5})$	(9, 9, 9, 9, 9)
Twiddle fractional bits $w_{f,tw}$	9
RTL data word-length DATA_W	24
RTL global fractional bits FRAC_W	18
MATLAB fixed-point 96-sample SQNR	41.45 dB

4 SDF FFT Architecture and Control Signals (Step 4–5)

4.1 Processing Element Control

The 32-point SDF FFT consists of five cascaded processing stages. The delay lengths are 16, 8, 4, 2, and 1 samples, respectively. A 5-bit counter `cnt[4:0]` is synchronized with the input sample index. The butterfly control signal `Ctrl_i` controls whether the PE performs butterfly operation or bypass operation. The selected control rule is listed in Table 3.

Table 3: Step 4: PE butterfly/bypass control signals.

Stage	Delay length	Control signal	Operation when control is 1
1	16	<code>Ctrl_1=cnt[4]</code>	Butterfly
2	8	<code>Ctrl_2=cnt[3]</code>	Butterfly
3	4	<code>Ctrl_3=cnt[2]</code>	Butterfly
4	2	<code>Ctrl_4=cnt[1]</code>	Butterfly
5	1	<code>Ctrl_5=cnt[0]</code>	Butterfly

TODO: Add Step 4 control signal waveform or timing table:
`step4_control_signal_table_or_waveform.png`.

Figure 9: TODO: Step 4 control signal verification.

4.2 Complex Multiplier Control

In the DIF SDF flow, twiddle multiplication is applied to the difference path. The twiddle factor of a 32-point FFT is

$$W_{32}^k = e^{-j2\pi k/32}.$$

When the phase index is zero, the twiddle factor is one, and the multiplier can be bypassed. Otherwise, the complex multiplier is enabled for the corresponding difference-path output.

Table 4: Step 5: Multiplier control rule by stage.

Stage	Twiddle phase step	Multiplier control
1	1	Enable when phase index $\neq 0$
2	2	Enable when phase index $\neq 0$
3	4	Enable when phase index $\neq 0$
4	8	Enable when phase index $\neq 0$
5	16	Bypass because phase is always 0 in the last butterfly group

TODO: Add Step 5 multiplier control table or waveform:
`step5_multiplier_control_table.png`.

Figure 10: TODO: Step 5 multiplier control verification.

5 Twiddle ROM32 Design (Step 6)

The ROM32 table must generate 16 FFT twiddle phases. Since sine and cosine values are symmetric, only the first-quadrant values from phase 0 to phase 8 are stored. The remaining phases are obtained by sign inversion and phase mirroring. The RTL twiddle value is represented as

$$W_{32}^k = \cos(2\pi k/32) - j \sin(2\pi k/32).$$

Table 5: First-quadrant ROM values before fixed-point scaling.

Phase k	$\cos(2\pi k/32)$	$\sin(2\pi k/32)$
0	1.0000	0.0000
1	0.9808	0.1951
2	0.9239	0.3827
3	0.8315	0.5556
4	0.7071	0.7071
5	0.5556	0.8315
6	0.3827	0.9239
7	0.1951	0.9808
8	0.0000	1.0000

TODO: Add ROM32 symmetry block diagram:
`step6_rom32_block_diagram.png`.

Figure 11: TODO: Step 6 ROM32 block diagram.

6 Bit-Reversal Ping-Pong Buffer Verification (Step 7)

The bit-reversal buffer uses two 32-word memory banks. While one bank writes incoming SDF outputs using bit-reversed addresses, the other bank reads data out in normal order. After one 32-sample symbol is completed, the read and write banks are swapped. This ping-pong access pattern allows continuous symbol-level reordering.

The verification input sequence is the bit-reversed index sequence

$$0, 16, 8, 24, 4, 20, 12, 28, \dots, 31,$$

and the expected output sequence is

$$0, 1, 2, \dots, 31.$$

[Figure placeholder: `step7_bit_reverse_waveform.png`]

Figure 12: Step 7: Behavioral simulation waveform of the ping-pong bit-reversal buffer.

TODO: Add waveform screenshot with `clk`, `rst_n`, `valid_in`, `SDFOutRe/SDFOutIm`, `BR0utRe/BR0utIm`, `wr_bank`, `rd_bank`, `wr_addr`, and `rd_addr`.

7 32-Point FFT Behavioral Simulation (Step 8)

The Step 8 behavioral simulation uses the quantized version of `FFTInput32.mat`. The testbench changes `FFTInRe`, `FFTInIm`, and `valid_in` only on the negative edge of `clk`. The DUT samples the inputs on the positive edge of `clk`, so the simulation avoids setup/hold ambiguity.

[Figure placeholder: step8_behav_waveform.png]

Figure 13: Step 8: Behavioral simulation waveform for one 32-point FFT symbol.

TODO: Add waveform screenshot with FFTInRe/FFTInIm, SDFOutRe/SDFOutIm, BROutRe/BROutIm, sdf_valid_out, and br_valid_out.

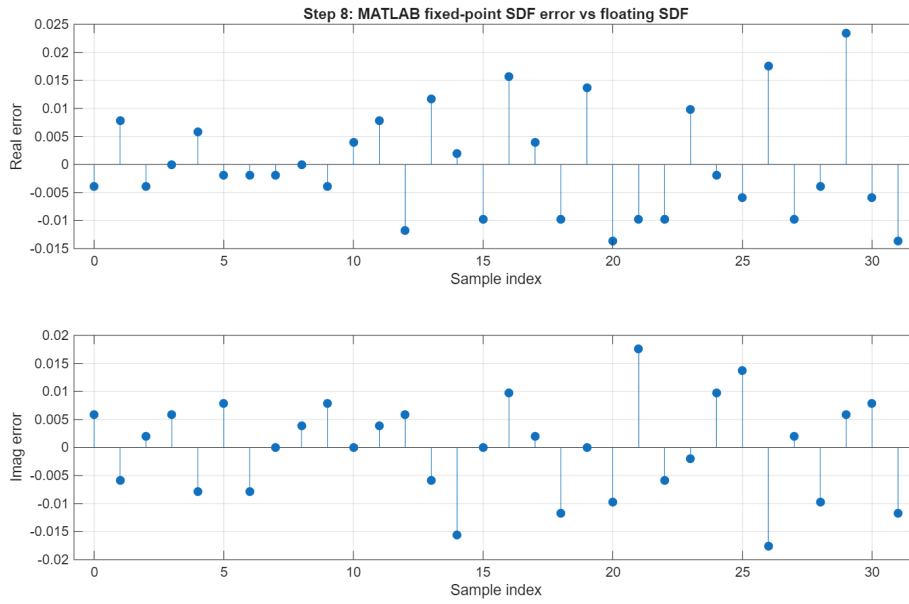


Figure 14: Step 8: MATLAB fixed-point SDF error compared with floating-point SDF output.

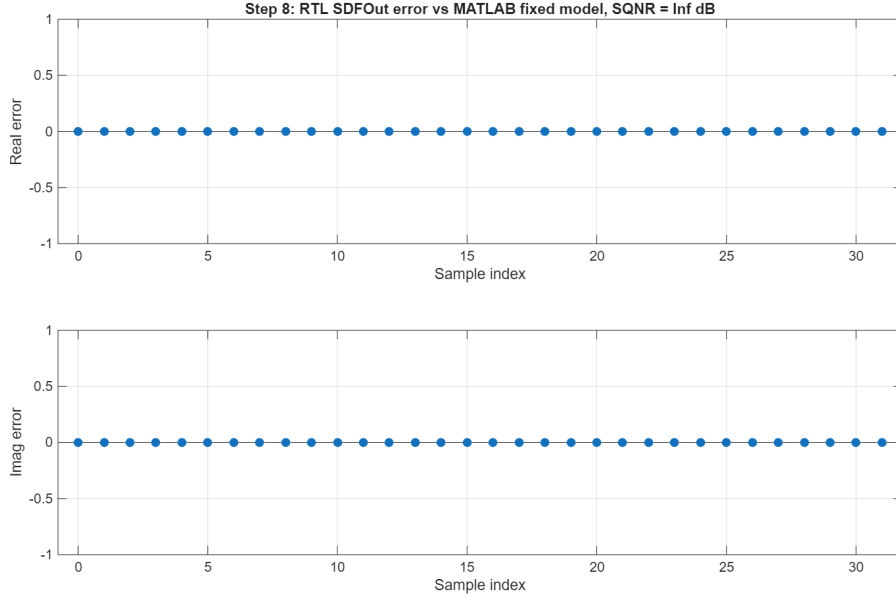


Figure 15: Step 8: RTL SDFOut error compared with MATLAB fixed-point golden output.

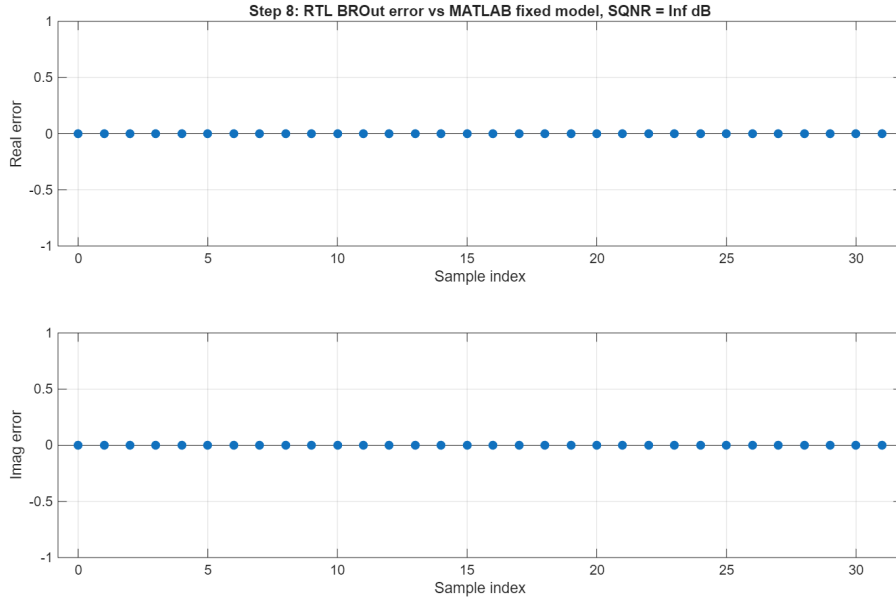


Figure 16: Step 8: RTL BR0ut error compared with MATLAB fixed-point golden output.

The RTL behavioral simulation matches the MATLAB fixed-point golden output bit-exactly for both SDFOut and BR0ut.

8 96-Sample Streaming Verification (Step 9)

In Step 9, three consecutive 32-point FFT symbols are fed into the design for a total of 96 samples. The SDF FFT core produces streaming outputs continuously. The bit-reversal

buffer introduces an additional symbol-level delay because a complete 32-sample block must first be written before it can be read out in normal order.

[Figure placeholder: step9_streaming_waveform.png]

Figure 17: Step 9: Behavioral simulation waveform for 96-sample streaming input.

TODO: Add waveform screenshot showing three input symbols and the corresponding SDFOut and BROut streaming outputs.

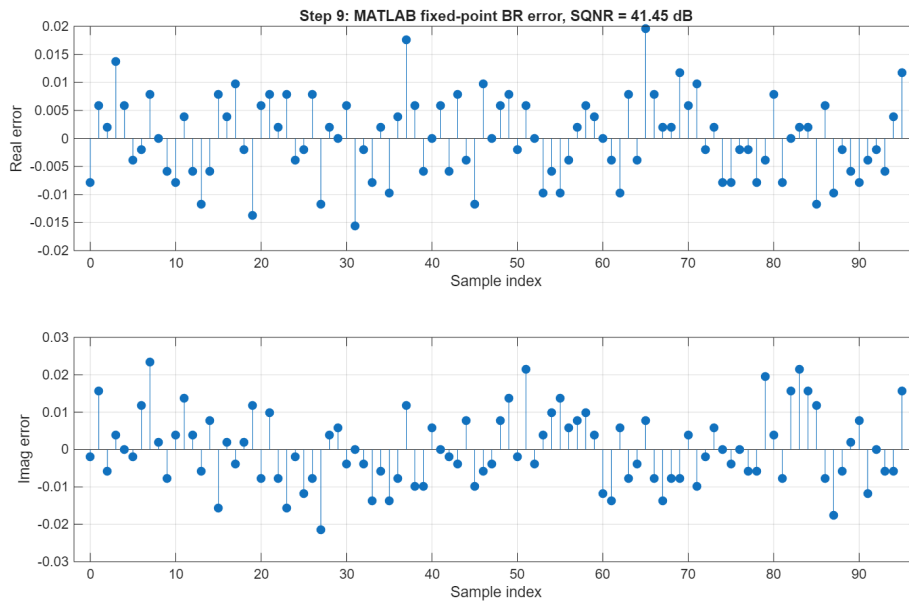


Figure 18: Step 9: MATLAB fixed-point normal-order output error for 96 samples.

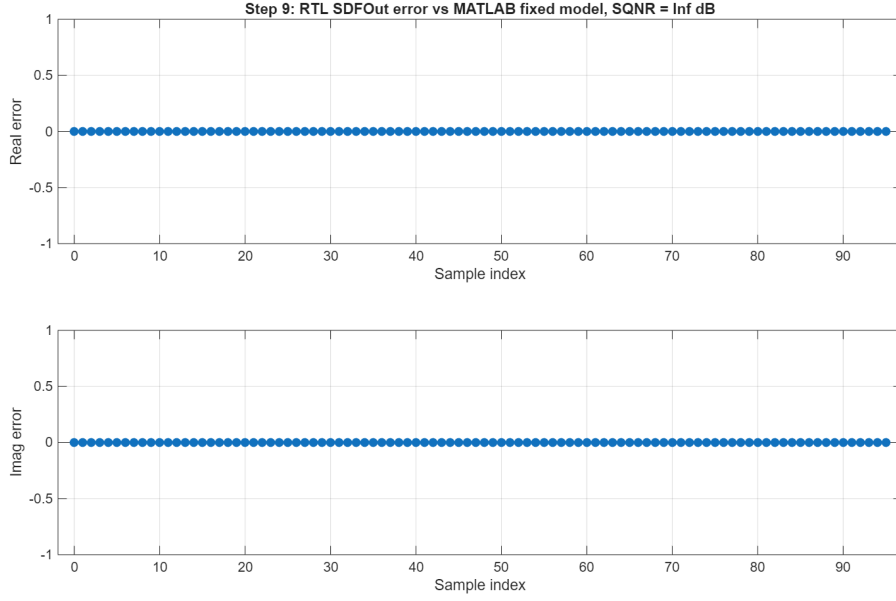


Figure 19: Step 9: RTL SDFOut error compared with MATLAB fixed-point golden output.

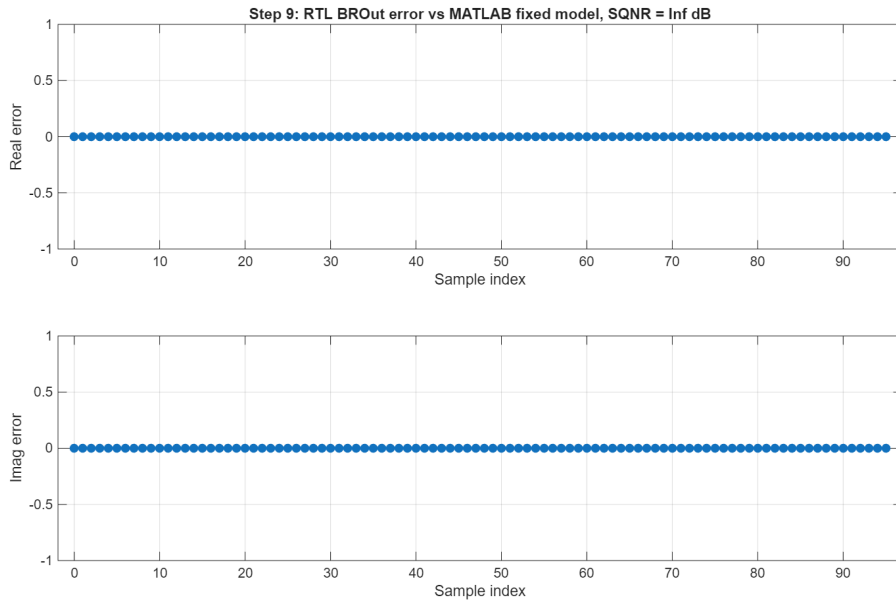


Figure 20: Step 9: RTL BR0ut error compared with MATLAB fixed-point golden output.

The MATLAB fixed-point model achieves an SQNR of 41.45 dB for the 96-sample stream. The RTL outputs are bit-exact with the MATLAB fixed-point golden model; therefore, the RTL-to-golden SQNR is infinite.

9 Synthesis Result without Internal Pipeline (Step 10)

In this step, D flip-flops are inserted at the input and output of the FFT processor, and the design is synthesized without internal pipeline registers. The purpose is to observe the critical path and report the maximum delay.

Table 6: Step 10: Baseline synthesis timing result.

Item	Value
Tool	TODO
Target library	TODO
Clock period	TODO
Maximum delay / critical path	TODO
Timing result	TODO

TODO: Add Step 10 synthesis timing report screenshot:
`step10_synthesis_timing_report.png`.

Figure 21: TODO: Step 10 synthesis timing report.

10 Pipelined FFT and Post-Synthesis Simulation (Step 11)

Internal pipeline registers are inserted to accelerate the design. The target operating frequency is

$$f_s = 300 \text{ MHz},$$

so the clock period is set to

$$T_{clk} = \frac{1}{f_s} = 3.333 \text{ ns}.$$

Table 7: Step 11: Pipelined synthesis timing result.

Item	Value
Target frequency	300 MHz
Clock period	3.333 ns
Worst negative slack	TODO
Timing result	TODO

[Figure placeholder: step11_postsyn_waveform.png]

Figure 22: Step 11: Post-synthesis simulation waveform at 300 MHz.

[Figure placeholder: step11_postsyn_real_imag_error.png]

Figure 23: Step 11: Real and imaginary errors for 96 post-synthesis output samples.

TODO: Add Step 11 timing summary screenshot:
step11_timing_summary.png.

Figure 24: TODO: Step 11 timing summary.

11 Conclusion

The MATLAB floating-point SDF FFT model is verified against MATLAB `fft()` with a maximum error of 5.551×10^{-16} . The selected fixed-point fractional word-lengths satisfy the 35 dB SQNR requirement, and the 96-sample fixed-point model achieves 41.45 dB SQNR. The behavioral RTL simulation matches the MATLAB fixed-point golden model bit-exactly for both the 32-point `FFTInput32.mat` test and the 96-sample streaming test.

The remaining report items are synthesis and post-synthesis screenshots for Step 10 and Step 11, together with the corresponding timing numbers.